

API SECURITY RISK ANALYSIS REPORT

Prepared for
Future Interns

Prepared by
ORITONDA NEMALALE
CIN: FIT/MAY26/CS8151
Date: June 1, 2026

EXECUTIVE SUMMARY

An API Security Risk Analysis was conducted on the JSONPlaceholder demo API (<https://jsonplaceholder.typicode.com>) on June 3, 2026. The analysis focused on identifying common API security risks using read-only, ethical testing methods following the OWASP API Security Top 10 framework.

KEY FINDINGS

TOTAL VULNERABILITIES DETECTED:	4
CRITICAL SEVERITY:	2
MODERATE SEVERITY:	2
MINIMAL SEVERITY:	0

RISKS IDENTIFIED

RISK TYPE:	SEVERITY:
BROKEN AUTHENTICATION	CRITICAL
BROKEN OBJECT LEVEL AUTH	CRITICAL
EXCESSIVE DATA EXPOSURE	MODERATE
MISSING RATE LIMITING	MODERATE

TOOLS USED

TARGET API:	https://jsonplaceholder.typicode.com
ANALYSIS SCOPE:	READ-ONLY / ETHICAL TESTING
AUDIT DATE:	JUNE 3, 2026
SECURITY STANDARDS:	OWASP API TOP 10 (2023)

The PostMan Tool was used to conduct the API analysis.

ETHICAL TESTING PARAMETERS

LIMITED TO PUBLIC DEMONSTRATION ENDPOINTS
NON-DESTRUCTIVE RETRIEVAL OPERATIONS ONLY
EXCLUDES PENETRATION OR CIRCUMVENTION EFFORTS
STRICT ADHERENCE TO NON-DISRUPTIVE SCANNING

Finding #1: Broken Authentication

RISK SEVERITY: HIGH

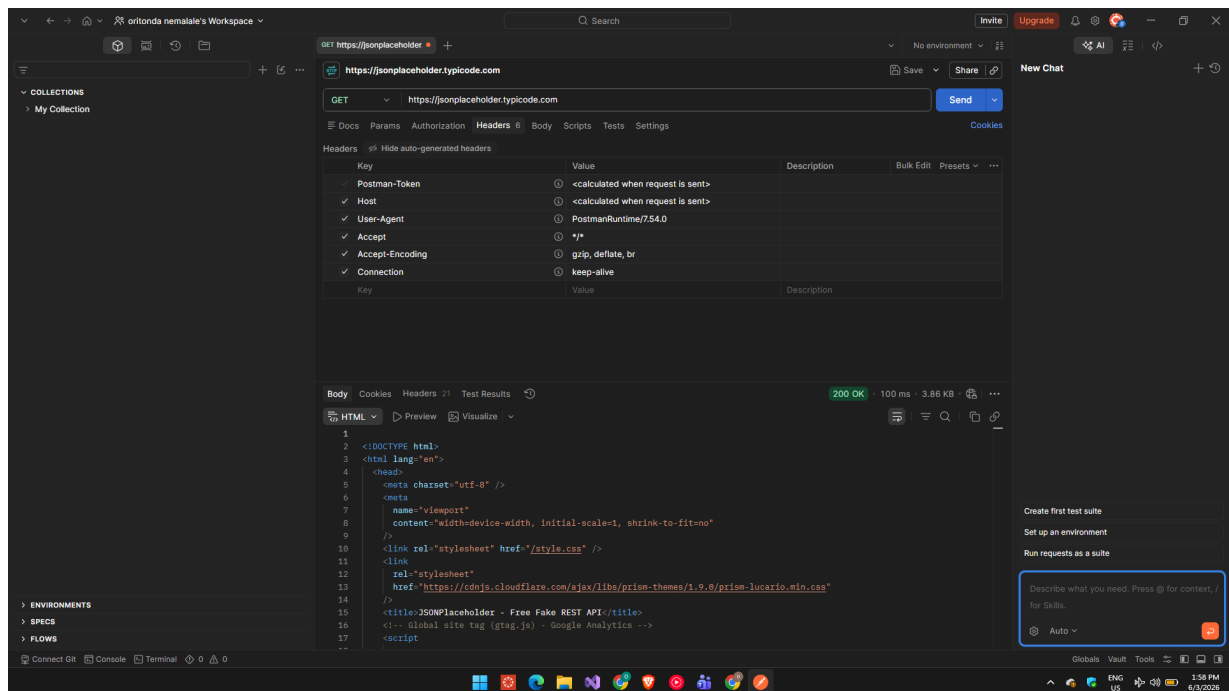
DESCRIPTION:

The API endpoints can be accessed without any form of authentication. No API keys, tokens, or credentials are required to retrieve user data.

EVIDENCE:

The GET request to /posts returned a 200 OK response with full post data without any authentication headers. Anyone on the internet can access all API data without providing any identification. An attacker can steal all user information in a single request. This is a critical security failure for any production API handling real user data.

Implement strong authentication mechanisms such as API keys, OAuth2, or JWT tokens. All endpoints that handle sensitive data must require valid authentication credentials.



Finding #2: Broken Object Level Authorization

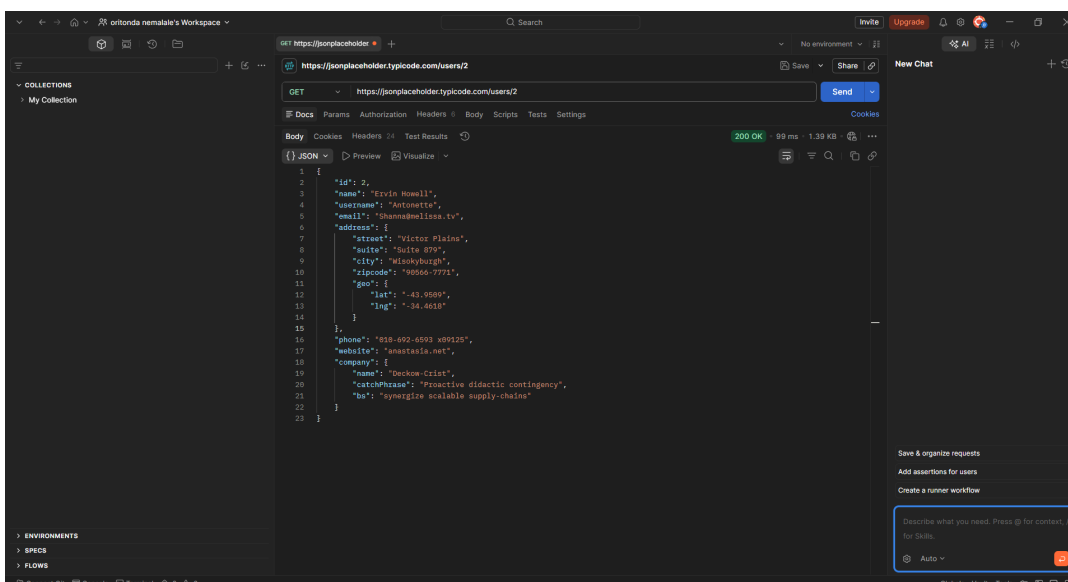
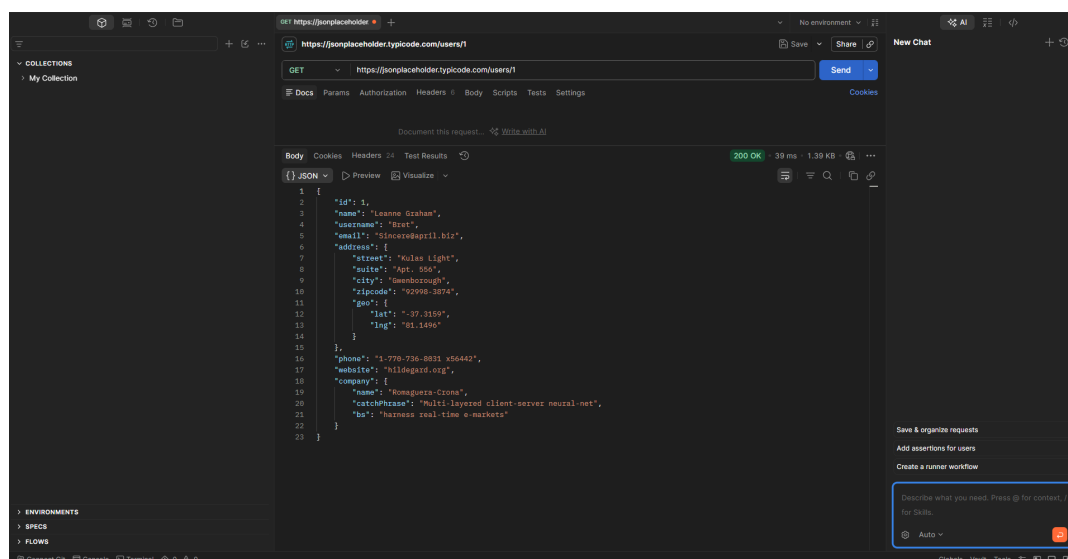
RISK SEVERITY: HIGH

DESCRIPTION:

The API allows unauthorized access to resources belonging to other users by simply modifying object identifiers in the URL. No authorization checks are performed before returning data.

EVIDENCE :

Changing userId from 1 to 2 successfully returned a different user's data with no authorization check. An attacker can access any user's data by guessing or enumerating user IDs. This allows large-scale data scraping of all user information. For a business API, this would expose customer data, leading to privacy violations and regulatory fines. Implement proper authorization checks on every endpoint that accesses resources by ID. The API must verify that the authenticated user has permission to access the requested resource before returning any data.



FINDING #3: Excessive Data Exposure

RISK SEVERITY: MEDIUM

DESCRIPTION:

The API returns more data than necessary, exposing sensitive fields that are not required for the client's intended purpose.

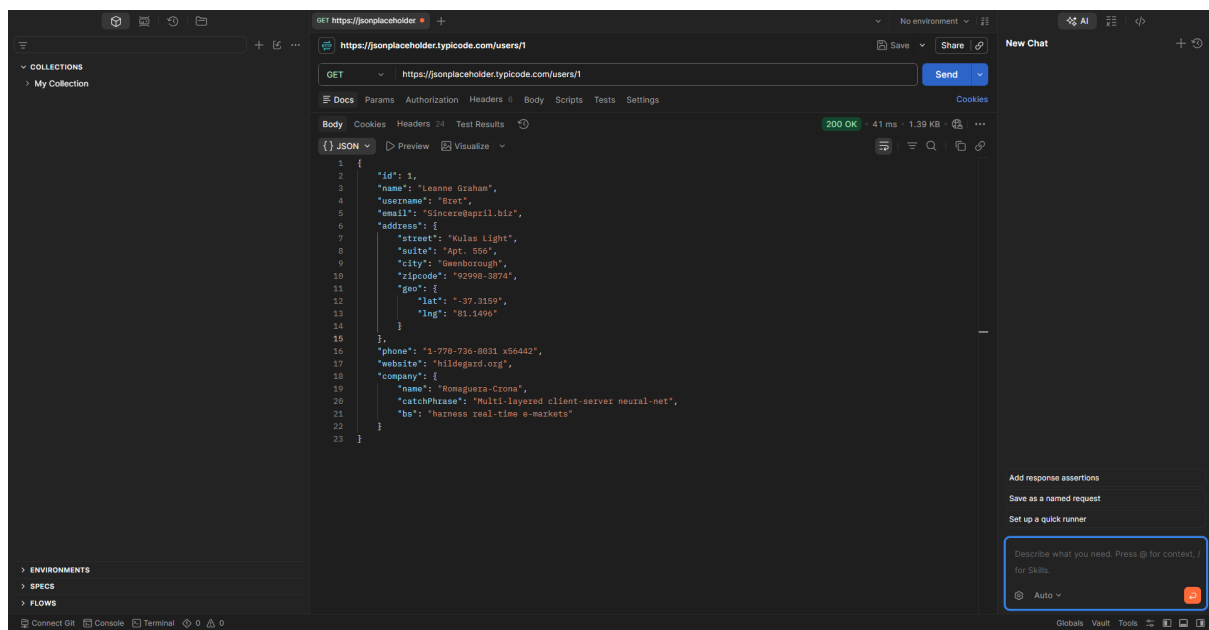
EVIDENCE:

GET /users/1 returned:

- id, name, username, email
- address (street, suite, city, zipcode, geo with lat/lng)
- phone, website
- company (name, catchPhrase, bs)

Exposing excessive data increases the damage of a security breach. If an attacker gains access, they receive more sensitive information than necessary. This violates the principle of least privilege and can lead to privacy compliance issues (GDPR, etc

Implement field-level filtering so the API only returns fields explicitly requested by the client. Use Data Transfer Objects (DTOs) to control exactly what data is exposed.



Finding #4: Missing Rate Limiting

RISK SEVERITY: MEDIUM

DESCRIPTION:

The API does not implement any rate limiting controls. Multiple requests can be sent in rapid succession without restriction.

EVIDENCE:

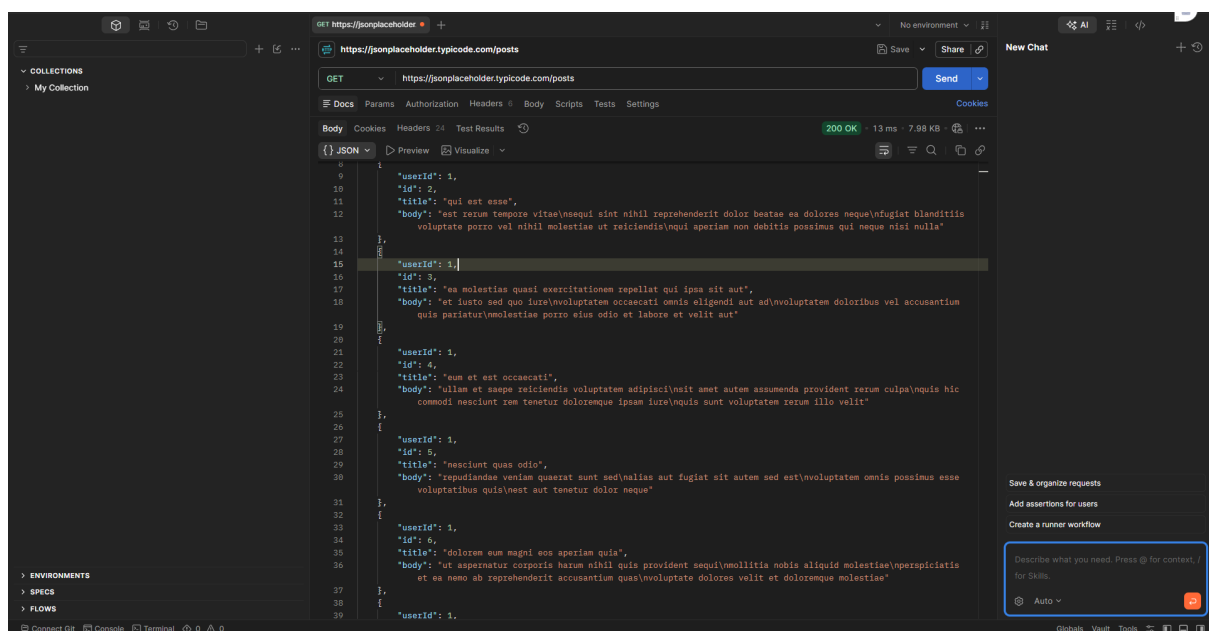
Multiple rapid GET requests to /posts were sent in succession. All returned 200 OK. No 429 (Too Many Requests) status codes were observed.

Without rate limiting, an attacker can:

- Perform brute-force attacks to guess passwords
- Scrape all data from the API at high speed
- Launch Denial-of-Service (DoS) attacks by overwhelming the server
- Incur excessive costs if the API is paid per request

Implement rate limiting at the API gateway or application level:

- Limit requests per IP address per time window
- Limit requests per authenticated user
- Return HTTP 429 status code when limits are exceeded



CONCLUSION & RECOMMENDATIONS

The JSONPlaceholder demo API contains multiple common API security vulnerabilities. While acceptable for a test API, these findings highlight critical security gaps that would be unacceptable in a production environment

SUMMARY OF FINDINGS:

Broken Authentication
Broken Object Level Authorization
Excessive Data Exposure
Missing Rate Limiting

KEY INSIGHT:

The API operates on a "trust by default" model, assuming that Anyone who can reach an endpoint should have access to the data. This violates every major principle of secure API design.

FINAL RECOMMENDATIONS

For API Development Teams:

1. IMPLEMENT AUTHENTICATION FIRST

No API endpoint should expose sensitive data without verifying the identity of the requester.

2. ENFORCE AUTHORIZATION ON EVERY REQUEST

Always verify that the authenticated user has permission to access the specific resource they are requesting.

3. MINIMIZE DATA EXPOSURE

Return only the data the client explicitly needs. Never return entire database objects.

4. DEPLOY RATE LIMITING

Protect APIs from brute-force attacks and DoS by implementing rate limits at multiple levels (IP, user, endpoint).

5. FOLLOW OWASP API SECURITY TOP 10

Use the framework as a checklist during API design and testing.